

Entrega 4 – Desenvolvimento dos Cadastros

Clientes • Equipamentos • Usuários • Integração com Banco • CRUD Completo

Equipe de Desenvolvimento

Alex Alves	github.com/alexx-al3
Andrey Gabriel	github.com/Andrey-Gabriel-Ben
Orientador: Prof. Rodrigo Costa	github.com/razevedocosta

1. Módulo de Clientes

Implementado em **Cliente.java** (Model), **ClienteDao.java** (DAO) e **ClienteControler.java** (Controller), com interface em **TelaCadastroCliente.java**.

Funcionalidade	Método	Observação
Cadastro de Cliente	ClienteControler.cadastrarCliente()	Valida CPF, telefone, e-mail e unicidade antes do INSERT
Busca por CPF	ClienteDao.buscarPorCpf()	Retorna objeto Cliente ou null; base para abertura de OS
Busca por Nome	ClienteDao.buscarPorNome()	Query com ILIKE para busca parcial case-insensitive

Validações implementadas no ClienteControler:

- CPF obrigatório com validação de dígito verificador
- CPF único – consulta ao banco via buscarPorCpf() antes do INSERT
- Telefone obrigatório com mínimo de 11 dígitos
- Nome e e-mail obrigatórios
- Feedback visual inline usando FlatLaf (JComponent.putClientProperty 'JComponent.outline', 'error')

2. Módulo de Equipamentos

Implementado em **Equipamento.java** (Model), **EquipamentoDao.java** (DAO) e **TelaCadastroEquipamento.java** (View).

Funcionalidade	Método	Observação
Cadastro de Equipamento	EquipamentoDao.salvar()	Vinculado a cliente; retorna ID gerado (RETURN_GENERATED_KEYS)
Listagem por Cliente	EquipamentoDao.listarPorCliente()	Popula o JComboBox na tela de abertura de OS

- Equipamento obrigatoriamente vinculado a um cliente (id_cliente NOT NULL)
- Número de série único (constraint UNIQUE no banco + validação prévia)
- Tipo, marca, modelo e descrição do defeito são campos obrigatórios

3. Módulo de Usuários

Implementado em **Usuario.java** (Model), **UsuarioDao.java** (DAO) e **UsuarioControler.java** (Controller), com telas **TelaLogin.java** e **TelaCadastroUsuario.java**.

Funcionalidade	Método	Observação
Cadastro de Usuário	UsuarioControler.cadastrarNovoUsuario()	Valida unicidade do login e criptografa senha com BCrypt antes do INSERT

Funcionalidade	Método	Observação
Autenticação	UsuarioDao.autenticarUsuario()	Busca por login; verifica hash com BCrypt.checkpw(); retorna Usuario ou null
Roteamento por Perfil	UsuarioController.efetuarLogin()	ATENDENTE → TelaAtendente; TECNICO → TelaMenuTecnico
Busca por Login	UsuarioDao.buscarPorLogin()	Verifica duplicidade antes do cadastro (case-insensitive)

Segurança implementada:

- Senhas armazenadas como hash BCrypt (nunca texto puro no banco)
- BCrypt.checkpw() para validação sem descriptografar
- Login único (constraint UNIQUE + verificação prévia no Controller)
- Senha mínima de 8 caracteres validada no Controller

4. Integração com Banco de Dados

A camada de acesso é centralizada em **ConexaoBanco.java**, que lê **Banco.properties** via ClassLoader e retorna conexões JDBC.

Recurso	Descrição
ConexaoBanco.getConexao()	Lê Banco.properties, registra driver PostgreSQL, retorna Connection
ConexaoBanco.fecharConexao()	Fecha conexão de forma segura (null-safe)
Try-with-resources	Todos os DAOs garantem fechamento automático de Connection e PreparedStatement
PreparedStatement	Toda query usa parâmetros '?' evitando SQL Injection
RETURN_GENERATED_KEYS	EquipamentoDao e OrdemServicoDao recuperam o ID gerado após INSERT